

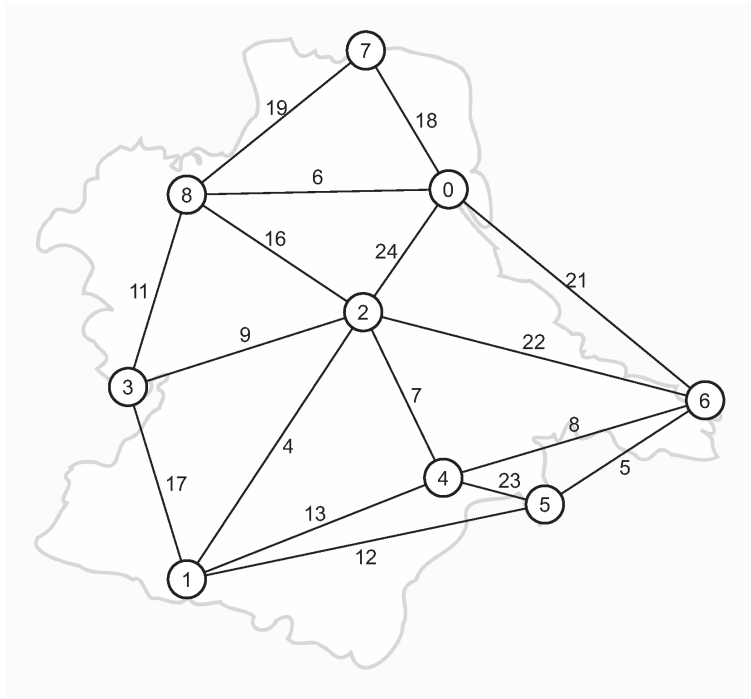


Figure 2 - Réseau simplifié du Listenbourg

La première partie du sujet s'intéresse à l'étude d'une base de données du réseau mise à disposition des fournisseurs d'accès (informatique de tronc commun, langage SQL). La deuxième partie introduit et étudie quelques propriétés de l'arbre couvrant de poids maximal (option informatique). La troisième partie s'intéresse à la recherche de l'arbre couvrant de poids maximal en utilisant l'algorithme de Borůvka (option informatique, langage OCaml). La quatrième partie se propose de montrer que la bande passante limite d'un réseau correspond au poids de l'arête de poids minimal de l'arbre couvrant de poids maximal du graphe (option informatique). La cinquième partie propose de modéliser la procédure de répartition de la gestion du réseau comme un jeu à deux joueurs (informatique de tronc commun, langage Python).

Les cinq parties sont essentiellement indépendantes dans leur traitement mais des notions utiles à la résolution d'une partie peuvent être introduites dans les parties précédentes. La **partie I** est indépendante des quatre autres. La **partie II** introduit des notions utiles à la résolution des **parties III** et **IV**. La **partie IV** introduit des concepts utilisés dans la **partie V**.

Dans tout le sujet, il est toujours admis d'utiliser un résultat ou un programme correspondant à une question précédente, même si cette question n'a pas été résolue.

- Q1.** Au vu de la situation modélisée, proposer des clés primaires pour les tables **villes** et **liaisons**, en justifiant succinctement. Identifier les clés étrangères présentes dans ce schéma relationnel.
- Q2.** Pour chacune des questions suivantes on demande d'écrire une requête SQL portant sur le schéma relationnel proposé qui fonctionnerait quel que soit le contenu de la base de données et pas uniquement sur l'exemple de contenu proposé à la **figure 3**.
- (a) Écrire une requête SQL qui donne les noms des villes comportant strictement plus de cinq millions d'habitants;
 - (b) Écrire une requête SQL qui donne la bande passante moyenne des liaisons incidentes à la ville d'identifiant 2;
 - (c) Écrire une requête SQL qui donne les noms des deux villes reliées par la liaison de bande passante maximale. On suppose que toutes les liaisons ont une bande passante différente.
- Q3.** Déterminer le résultat de la requête SQL suivante sur l'exemple de contenu de la base de données représentée à la **figure 3**.

```
SELECT nom, COUNT(*) AS d
FROM villes
JOIN (SELECT id1 AS x, id2 FROM liaisons UNION SELECT id2 AS x, id1 FROM liaisons)
ON id = x
GROUP BY id
HAVING d >= 4
ORDER BY id ASC
```

Partie II - Arbre couvrant de poids maximal

II.1 - Acyclicité, connexité et arbres

Un **graphe** (non orienté) est un couple $G = (S, A)$ formé d'un ensemble S fini non vide de **sommets** et d'un ensemble A d'**arêtes** constitué de parties de S de cardinal 2. On note $|S|$ le nombre de sommets de G et $|A|$ le nombre d'arêtes de G . Les voisins d'un sommet $x \in S$ sont notés $V(x)$ et leur nombre $d(x) = |V(x)|$ est le **degré** de x .

Un chemin dans un graphe est une suite $c = x_0 x_1 \dots x_n$ de sommets avec $\forall i \in \llbracket 0, n-1 \rrbracket, \{x_i, x_{i+1}\} \in A$. La longueur $|c|$ d'un tel chemin est $n \in \mathbb{N}$. Un chemin est **élémentaire** si tous les sommets empruntés par ce chemin sont deux à deux distincts. Un chemin est **simple** si toutes les arêtes empruntées par ce chemin sont deux à deux distinctes.

Un **cycle** est un chemin simple $x_0 x_1 \dots x_n$ avec $n \geq 3$ et $x_0 = x_n$. Un graphe est **acyclique** s'il ne comporte pas de cycle.

Un graphe est **connexe** si deux sommets quelconques sont toujours reliés par un chemin. Une **composante connexe** est un ensemble $C \subseteq S$ de sommets dont tous les couples de sommets sont reliés par au moins un chemin.

Un **arbre** est un graphe connexe acyclique.

Si $a = \{x, y\}$ est une partie de S de cardinal 2, on note $G + a$ le graphe $(S, A \cup \{a\})$, c'est-à-dire le graphe G auquel on a ajouté l'arête a et $G - a$ le graphe $(S, A \setminus \{a\})$, c'est-à-dire le graphe G auquel on a retiré l'arête a .

Q4. Montrer que si $G = (S, A)$ est un graphe connexe et que si $a = \{x, y\} \in A$ est une arête de G appartenant à un cycle de G , alors le graphe $G - a$ est encore connexe.

On admet la proposition suivante :

Proposition 1 : relation entre $|S|$ et $|A|$ pour les graphes connexes ou acycliques

Soit $G = (S, A)$ un graphe :

- (a) si G est connexe alors $|A| \geq |S| - 1$;
- (b) si G est acyclique alors $|A| \leq |S| - 1$.

Q5. Montrer que si $G = (S, A)$ est connexe et que $|A| = |S| - 1$ alors G est un arbre.

II.2 - Sous-graphe et arbre couvrant

Dans tout ce sujet un *sous-graphe* d'un graphe G est un graphe $G' = (S, A')$ avec $A' \subseteq A$. Notons qu'un sous-graphe de G comporte exactement les mêmes sommets que G . On identifie un sous-graphe $G' = (S, A')$ avec le sous-ensemble d'arêtes $A' \subseteq A$. Un *arbre couvrant* de G est un sous-graphe de G qui est un arbre.

Q6. Montrer qu'un graphe $G = (S, A)$ est connexe si et seulement s'il admet un arbre couvrant.

Q7. Soit $T = (S, A')$ un arbre couvrant de G avec $A' \subsetneq A$ et $a \in A \setminus A'$.

- (a) Justifier que $T + a$ n'est pas acyclique.
- (b) Soit $a' \in A'$ une arête d'un cycle de $T + a$. Montrer que $T + a - a'$ est un arbre couvrant de G .

II.3 - Graphes pondérés et arbre couvrant de poids maximal

Un *graphe pondéré* est un triplet (S, A, p) où (S, A) est un graphe et $p : A \rightarrow \mathbb{R}$ une fonction de pondération des arêtes. Si $\{x, y\} \in A$ est une arête, $p(\{x, y\})$ est appelé le *poids* de cette arête. Le poids d'un graphe pondéré est la somme des poids de ses arêtes :

$$p(G) = \sum_{a \in A} p(a).$$

On étend la notion de sous-graphe aux graphes pondérés de la manière suivante : un sous-graphe d'un graphe pondéré $G = (S, A, p)$ est un graphe pondéré $G' = (S, A', p_{A'})$ où $A' \subseteq A$ et où $p_{A'}$ est la restriction de p à A' . On se permettra de ne pas systématiquement indiquer la fonction de pondération.

Le plan « Réseau pour Tous » du gouvernement a permis d'interconnecter toutes les villes du Listenbourg. On considérera donc en général un graphe pondéré $G = (S, A, p)$ non orienté *connexe*, avec la fonction de pondération correspondant à la bande passante associée à chaque liaison. Chaque province ayant opté pour son propre câblage, deux villes ne sont jamais reliées par le même type de câblage et la bande passante de deux liaisons est *toujours* différente. Ceci se traduit donc par une fonction de pondération p injective : dans toute la suite du sujet, les poids des arêtes de G sont toujours deux à deux distincts.

On se propose de démontrer la **proposition 2** suivante :

Proposition 2 : existence et unicité de l'arbre couvrant de poids maximal

Soit $G = (S, A, p)$ un graphe *connexe* muni d'une fonction de pondération *injective*. Alors, il existe un unique arbre couvrant de poids maximal de G . On note $T^* = (S, A^*)$ cet arbre couvrant.

Q8. Montrer l'existence d'un arbre couvrant de poids maximal de G .

Q9. Pour montrer l'unicité, on se propose de procéder par l'absurde. Supposons l'existence de deux arbres couvrants de poids maximal $T_1 = (S, A_1)$ et $T_2 = (S, A_2)$ différents. Considérons une arête de poids maximal parmi les arêtes qui appartiennent à un seul des deux arbres, c'est-à-dire une arête de poids maximal dans $(A_1 \setminus A_2) \cup (A_2 \setminus A_1)$, qui est bien non vide puisque $A_1 \neq A_2$. Sans perte de généralité, on peut noter a_2 une telle arête et supposer que $a_2 \in A_2 \setminus A_1$.

- (a) Montrer qu'il existe une arête $a_1 \in A_1 \setminus A_2$ sur un cycle de $T_1 + a_2$.
- (b) Conclure en considérant le sous-graphe $T_1 + a_2 - a_1$.

Partie III - Recherche d'un arbre couvrant de poids maximal

III.1 - Rappels et fonctions élémentaires en OCaml

On rappelle que, dans le langage OCaml :

- on peut représenter des tableaux par le type `'a array` ;
- si `e` est une expression de type `'a`, l'expression `Array.make n e` permet de créer un tableau de type `'a array` de $n \in \mathbb{N}$ cases dont toutes les cases contiennent la valeur de l'expression `e` ;

`Array.make : int -> 'a -> 'a array`

- si `a` est un tableau de type `'a array` de taille $n \in \mathbb{N}$, l'expression `Array.length a` permet d'obtenir la valeur n ;

`Array.length : 'a array -> int`

- si $0 \leq i < n$, l'expression `a.(i)` s'évalue en la valeur contenue à la case `i` du tableau `a` et si `e` est une expression de type `'a`, l'expression `a.(i) <- e` permet de remplacer la valeur contenue à la case `i` du tableau `a` par la valeur de l'expression `e` ;
- la fonction `List.iter` a le comportement suivant : si `f` est une fonction de type `'a -> unit`, alors l'expression `List.iter f [a1; a2; ...; an]` est équivalente à `f a1; f a2; ...; f an` ;

`List.iter : ('a -> unit) -> 'a list -> unit`

- une boucle `for i = a to b do ... done` permet de faire évoluer une variable `i` entre les valeurs entières `a` et `b` **includes** ;
- il existe deux fonctions `max` et `min` de type `'a -> 'a -> 'a`.

Q10. Écrire une fonction `max_tab : int array -> int` telle que, pour un tableau d'entiers `a` de taille $n \geq 1$, `max_tab a` renvoie la valeur du plus grand entier contenu dans `a`. On attend une complexité linéaire en n , ce que l'on justifiera en une phrase.

III.2 - Représentation des graphes en OCaml

On représente un graphe pondéré $G = (S, A, p)$, avec $S = \llbracket 0, n - 1 \rrbracket$ et $n \in \mathbb{N}^*$ par un tableau de listes d'adjacence pondérées.

```
type graphe = (int * float) list array
```

Si g représente un graphe et si $0 \leq i < n$ est un sommet, alors $g.(i)$ contient la liste des couples $\left\{ (j, p((i, j))) \right\}_{j \in \mathcal{V}(i)}$, c'est-à-dire la liste des couples de la forme (j, w) avec $w = p((i, j))$ et j voisin de i . Remarquons que puisque les graphes sont non orientés, si i est dans la liste d'adjacence de j , alors j est dans la liste d'adjacence de i avec le même poids.

Par exemple, si g est le graphe de la **figure 2**, on a $g.(0)$ qui vaut par exemple :

```
[(2, 24.0); (6, 21.0); (7, 18.0); (8, 6.0)]
```

Le nombre de sommets du graphe correspond donc à la taille du tableau de listes d'adjacences :

```
let nb_sommets g = Array.length g
```

Q11. Donner une valeur possible pour $g.(7)$.

On représente une arête par un triplet (w, i, j) où $w = p((i, j))$ et où $i < j$. Remarquons que chaque arête est ainsi représentée exactement une fois et qu'une arête commence par son poids.

Q12. Écrire en OCaml une fonction `ajoute_arete : graphe -> (float * int * int) -> unit` telle que `ajoute_arete g (w, i, j)` permet d'ajouter au graphe g l'arête (w, i, j) . On suppose que cette arête n'était pas déjà présente et la complexité doit être en temps constant.

Q13. Écrire en OCaml une fonction `toutes_les_aretes : graphe -> (float * int * int) list` qui permet d'obtenir la liste des arêtes d'un graphe avec la convention ci-dessus. On attend une complexité linéaire en $|S| + |A|$, ce que l'on justifiera rapidement.

Si $a1$ et $a2$ sont deux arêtes de type `float * int * int`, l'expression `a1 < a2` compare les arêtes pour l'ordre lexicographique des triplets donc pour la valeur du poids de l'arête en premier. Comme dans ce sujet on suppose toujours que tous les poids sont deux à deux distincts, on peut directement utiliser l'ordre OCaml `<` sur les arêtes pour comparer les poids des arêtes. Ainsi par exemple, `min a1 a2` s'évalue en l'arête de poids minimal entre $a1$ et $a2$.

Q14. Écrire une fonction `min_arete : (float * int * int) list -> (float * int * int)` de manière récursive en OCaml qui renvoie l'arête de poids minimal d'une liste d'arêtes supposée non vide. Quelle est sa complexité ?

III.3 - Recherche des composantes connexes

Pour un graphe $G = (S, A, p)$ avec $S = \llbracket 0, n - 1 \rrbracket$, on cherche à construire un tableau c de taille n représentant les composantes connexes du graphe. Les composantes connexes sont indexées entre 0 et $p - 1$ où p est le nombre de composantes connexes du graphe. Pour $0 \leq i < n$ la case $c.(i)$ comporte l'indice k de la composante connexe du sommet i (ou la valeur -1 si cette composante n'a pas encore été déterminée).

On considère la fonction `explorer : graphe -> int array -> int -> int -> unit` suivante pour un graphe g , un tableau c de taille n , un identifiant de composante connexe $0 \leq k < p$ et un sommet de départ s , vérifiant la convention suivante : « un sommet $0 \leq i < n$ a été visité si et seulement si $c.(i) \geq 0$ ».

```
1 let rec explorer g c k s =
2   if c.(s) < 0 then begin
3     c.(s) <- k;
4     List.iter (fun (v, _) -> explorer g c k v) g.(s)
5   end
```

Q15. Décrire précisément ce que réalise un appel à `explorer g c k s`.

Q16. Écrire en OCaml une fonction `composantes_connexes : graphe -> int array` telle que, pour un graphe g , `composantes_connexes g` s'évalue en le tableau c des indices des composantes connexes. Cette fonction doit avoir une complexité en $O(|S| + |A|)$ que l'on ne demande pas de justifier.

Q17. En déduire une fonction `est_connexe : graphe -> bool` telle que `est_connexe g` s'évalue à `true` si le graphe g est connexe et à `false` sinon.

III.4 - Algorithme de Borůvka

Dans toute cette sous-partie, on considère un graphe $G = (S, A, p)$ connexe avec une fonction de pondération p injective. On note $T^* = (S, A^*)$ l'unique arbre couvrant de poids maximal de G .

Considérons $H = (S, B)$ un sous-graphe acyclique de G , avec donc $B \subseteq A$. On considère $C \subseteq S$ une composante connexe de H . Une arête **sûre** pour C est une arête de A ayant exactement une extrémité dans C dont le poids est maximal parmi les arêtes qui ont exactement une extrémité dans C .

Par exemple, pour le sous-graphe H de la **figure 4** et pour la composante connexe $\{1, 4, 5, 6\}$ l'arête de poids 22 est sûre (maximale parmi les arêtes de poids 4, 7, 17, 21 et 22 qui sont celles ayant exactement une extrémité dans cette composante). Sauf si H est connexe, chaque composante connexe de H comporte exactement une arête sûre. Une même arête peut être sûre pour deux composantes connexes C et C' . Dans l'exemple de la **figure 4**, l'arête de poids 22 est également l'arête sûre pour la composante connexe $\{0, 2\}$.

Q18. Déterminer l'arête sûre pour la composante $\{3\}$ ainsi que celle pour la composante $\{7, 8\}$.

Q19. On suppose dans cette question que $H = (S, B)$ est un sous-graphe acyclique de l'arbre couvrant $T^* = (S, A^*)$ de poids maximal de G , et donc que $B \subseteq A^*$. Montrer que A^* contient toutes les arêtes sûres des composantes connexes de H .

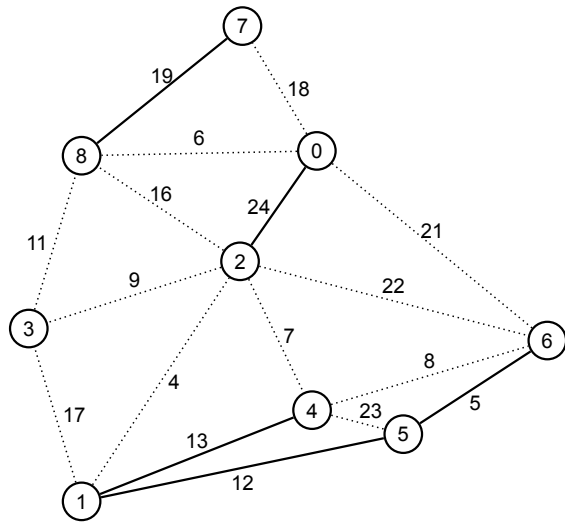


Figure 4 - Un sous-graphe H du graphe de la **figure 2**, comportant quatre composantes connexes. Les arêtes qui n'appartiennent pas au sous-graphe sont représentées en pointillés

L'idée de l'algorithme de Borůvka, dont le pseudocode est donné par l'**algorithme 1**, est de construire progressivement un sous-graphe acyclique $H = (S, B)$ de l'arbre couvrant de poids maximal $T^* = (S, A^*)$. Au départ $B = \emptyset$. À chaque étape, on ajoute au sous-graphe acyclique H en cours de construction toutes les arêtes sûres pour les composantes connexes de H .

Algorithme 1 : algorithme de Borůvka

Entrée : Graphe $G = (S, A, p)$ pondéré injectivement et connexe

Sortie : $H = (S, B)$ l'arbre couvrant de poids maximal de G

```

1  $H = (S, \emptyset)$ ;
2 tant que  $H$  n'est pas connexe faire
3   | ajouter à  $H$  toutes les arêtes sûres pour les composantes connexes de  $H$ ;
4 fin
```

Q20. Donner le déroulé de l'**algorithme 1** sur le graphe G de la **figure 2**. Indiquer à chaque étape et pour chaque composante connexe son arête sûre et indiquer l'ensemble des arêtes sûres ajoutées à chaque étape.

Q21. Montrer que l'algorithme de Borůvka termine.

Q22. Montrer que l'algorithme de Borůvka renvoie l'arbre couvrant de poids maximal $T^* = (S, A^*)$ de G .

On considère la fonction suivante qui prend en entrée un graphe g représentant le graphe $G = (S, A, p)$ connexe pondéré et un graphe h représentant un sous-graphe acyclique $H = (S, B)$ de G supposé non encore connexe.

```

1 let aretes_sures g h =
2   let c = composantes_connexes h in
3   let p = max_tab c + 1 in
4   let aretes = toutes_les_aretes g in
5   let sures = Array.make p (min_arete aretes) in
6   let traite a =
7     let (_, i, j) = a in
8     if c.(i) <> c.(j) then begin
9       sures.(c.(i)) <- max a sures.(c.(i));
10      sures.(c.(j)) <- max a sures.(c.(j));
11    end
12   in
13   List.iter traite aretes;
14   sures
```

Q23. Déterminer le type de cette fonction. Expliquer ce que réalise cette fonction et préciser la structure et le contenu de la valeur renvoyée. Déterminer sa complexité en fonction de $|A|$ en justifiant. On remarquera que d'après la **proposition 1**, puisque le graphe G est connexe, on a $|S| - 1 \leq |A|$ et donc qu'un $O(|S|)$ est un $O(|A|)$.

On suppose disposer d'une fonction :

```
ajoute_aretes_sures : graphe -> (float * int * int) array -> unit
```

telle que si h représente un sous-graphe acyclique de G et que $sures$ est la valeur renvoyée par `aretes_sures g h` alors `ajoute_aretes_sures h sures` ajoute à h toutes les arêtes contenues dans $sures$. On garantit que cette fonction est en complexité $O(|A|)$ et qu'elle n'ajoute qu'une seule fois une même arête à h même si cette arête est sûre pour deux composantes connexes différentes.

Q24. Écrire en OCaml une fonction `borukva` : `graphe -> graphe` telle que `borukva g` sur un graphe g s'évalue en un sous-graphe h de G correspondant à son arbre couvrant de poids maximal.

Q25. On s'intéresse maintenant à la complexité temporelle de cet algorithme.

- Montrer qu'à chaque passage dans la boucle **tant que** à la ligne 2 de l'**algorithme 1**, le nombre de composantes connexes de H est divisé au moins par 2.
- En déduire la complexité temporelle dans le pire cas de l'algorithme de Borůvka.
- Quelle est la complexité temporelle de cet algorithme dans le meilleur cas ?

Partie IV - Chemin de bande passante maximale

Dans cette partie, on considère un graphe pondéré $G = (S, A, p)$, non nécessairement connexe, mais dont la fonction de pondération est injective.